

Heaps Revision Notes

Striver SDE Sheet - Short approach notes for quick interview revision

Core Heap Mindset

- Use min heap when you need the smallest element quickly.
- Use max heap when you need the largest element quickly.
- For top k problems, keep a heap of size k.
- For running median, split numbers into two heaps.

1. Implement Min Heap

Pattern	Array + heapify
Approach	Store heap in an array. Insert at end and move up. Extract min by replacing root with last element and moving down.
Intuition	The heap property is restored locally using up-heapify or down-heapify.
Complexity	Insert $O(\log n)$, Extract $O(\log n)$, Get Min $O(1)$
Remember	parent = $(i-1)/2$, left = $2i+1$, right = $2i+2$.

2. Kth Largest / Kth Smallest Element

Pattern	Heap of size k
Approach	For kth largest, use min heap of size k. For kth smallest, use max heap of size k. Remove extra elements when size > k.
Intuition	The heap keeps only the best k candidates.
Complexity	$O(n \log k)$ time, $O(k)$ space
Remember	Top of heap becomes the kth required element.

3. Maximum Sum Combination

Pattern	Max heap + visited set
Approach	Sort both arrays. Start from largest pair sum $A[n-1]+B[n-1]$. Pop max pair, then push neighboring pairs $(i-1, j)$ and $(i, j-1)$.
Intuition	The maximum unexplored sum is always available at heap top.
Complexity	$O(k \log k)$ time after sorting, $O(k)$ space
Remember	Use set to avoid duplicate index pairs.

4. Find Median from Data Stream

Pattern	Two heaps
Approach	Use max heap for smaller half and min heap for greater half. Insert and rebalance so size difference is at most 1.
Intuition	Median is either the top of bigger heap or average of both tops.
Complexity	Insert $O(\log n)$, Find Median $O(1)$, Space $O(n)$
Remember	Max heap top $\leq$ min heap top should always hold.

5. Merge K Sorted Arrays

Pattern	Min heap of k pointers
Approach	Push first element of every array. Pop smallest, add to answer, then push next element from the same array.
Intuition	Heap keeps the smallest current candidate among all arrays.
Complexity	$O(N \log k)$ time, $O(k)$ space
Remember	Store value, array index, and element index.

6. K Most Frequent Elements

Pattern	HashMap + min heap
Approach	Count frequency using HashMap. Push unique elements into min heap by frequency. If heap size > k, remove smallest frequency.
Intuition	Heap keeps the k highest frequency elements.
Complexity	$O(n \log k)$ time, $O(n)$ space
Remember	For top k largest frequency, use min heap of size k.

Heap Pattern Summary

Problem Type	Pattern / Trick
Implement Heap	Array + heapify
Kth Largest / Smallest	Heap of size k
Maximum Sum Combination	Max heap + visited set
Median Stream	Two heaps
Merge K Sorted Arrays	Min heap with k pointers
K Most Frequent	Frequency map + min heap

Java Heap Syntax

- `PriorityQueue<Integer> minHeap = new PriorityQueue<>();`
- `PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());`
- For custom objects, pass a comparator based on value, frequency, or sum.